

Surveillance de la qualité des eaux dans les rivières des landes



Expression du besoin initial

La directive Cadre Européenne sur l'Eau (DCE) adopté le 23 octobre 2000, établit un cadre pour une politique communautaire dans le domaine de l'eau. L'objectif visé par les états membres est l'atteinte du bon état de l'ensemble des masses d'eau.

L'agence de l'eau Adour-Garonne a installé un réseau de surveillance des cours d'eau Landais.

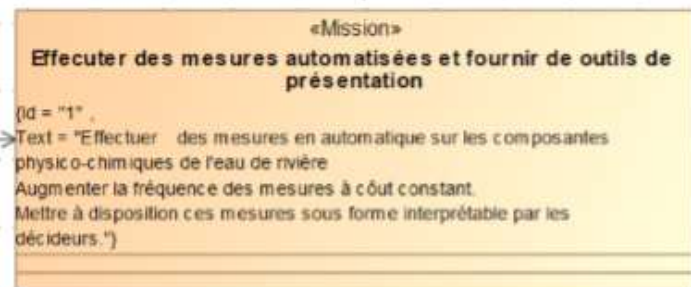
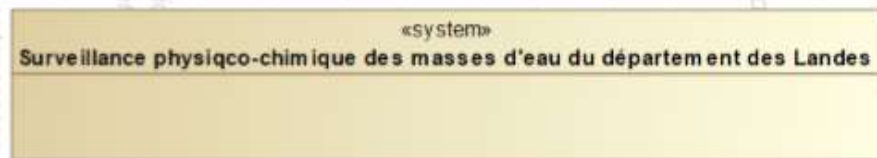
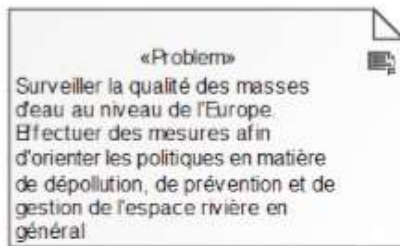
L'objectif est de surveiller l'évolution quantitative et qualitative de l'ensemble des cours d'eau du département des Landes. Cette surveillance permet de qualifier, selon des classes de qualité, l'état écologique des cours d'eau et de suivre leur évolution.

L'analyse des résultats doit permettre ensuite de préciser et réorienter, si nécessaire, les politiques d'investissement en matière de dépollution, de prévention et de gestion de l'espace rivière en général.

L'état du système: La surveillance physico-chimique a lieu tous les deux mois, les opérations sont réalisées par un binôme d'agents du SAGER pour prélever des échantillons. Ces échantillons sont remis au laboratoire pour l'analyse de nombreuses substances permettant d'évaluer la qualité des cours d'eau et de suivre leur évolution dans le temps. Les données recueillies sont présentées annuellement et font l'objet d'un rapport de synthèse édité tous les 5 ans. Le coût du système est de 180 000 euros/an.

On désire dans le cadre de ce projet:

- d'apporter des mesures à des fréquences plus importantes afin d'être plus pro-actifs
- de réduire le coût de fonctionnement du système.
- présenter les résultats en temps réel.
- automatiser les mesures.

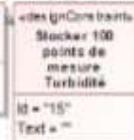
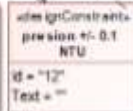
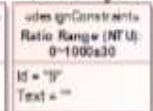
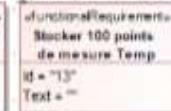
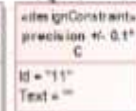
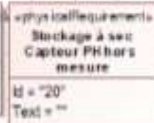
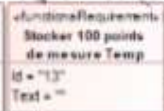
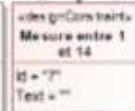
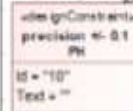
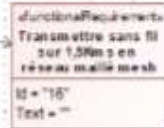
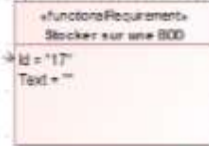
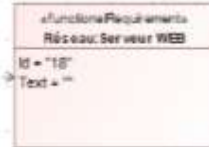
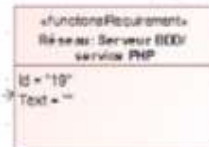
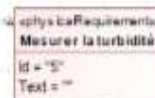
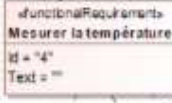
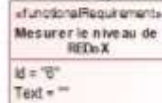
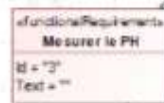
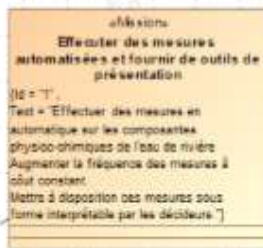


«satisfy»

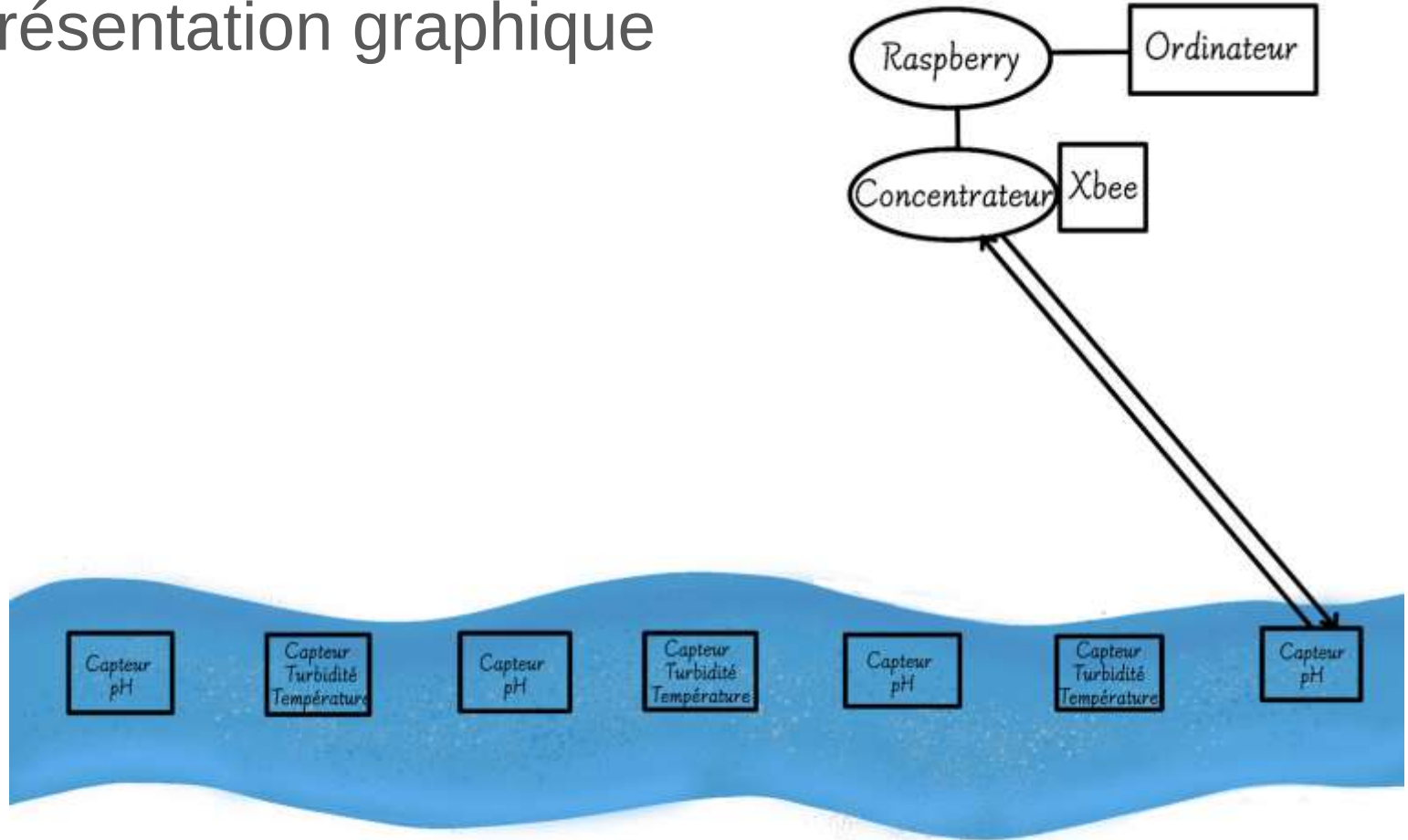
«deriveReq»

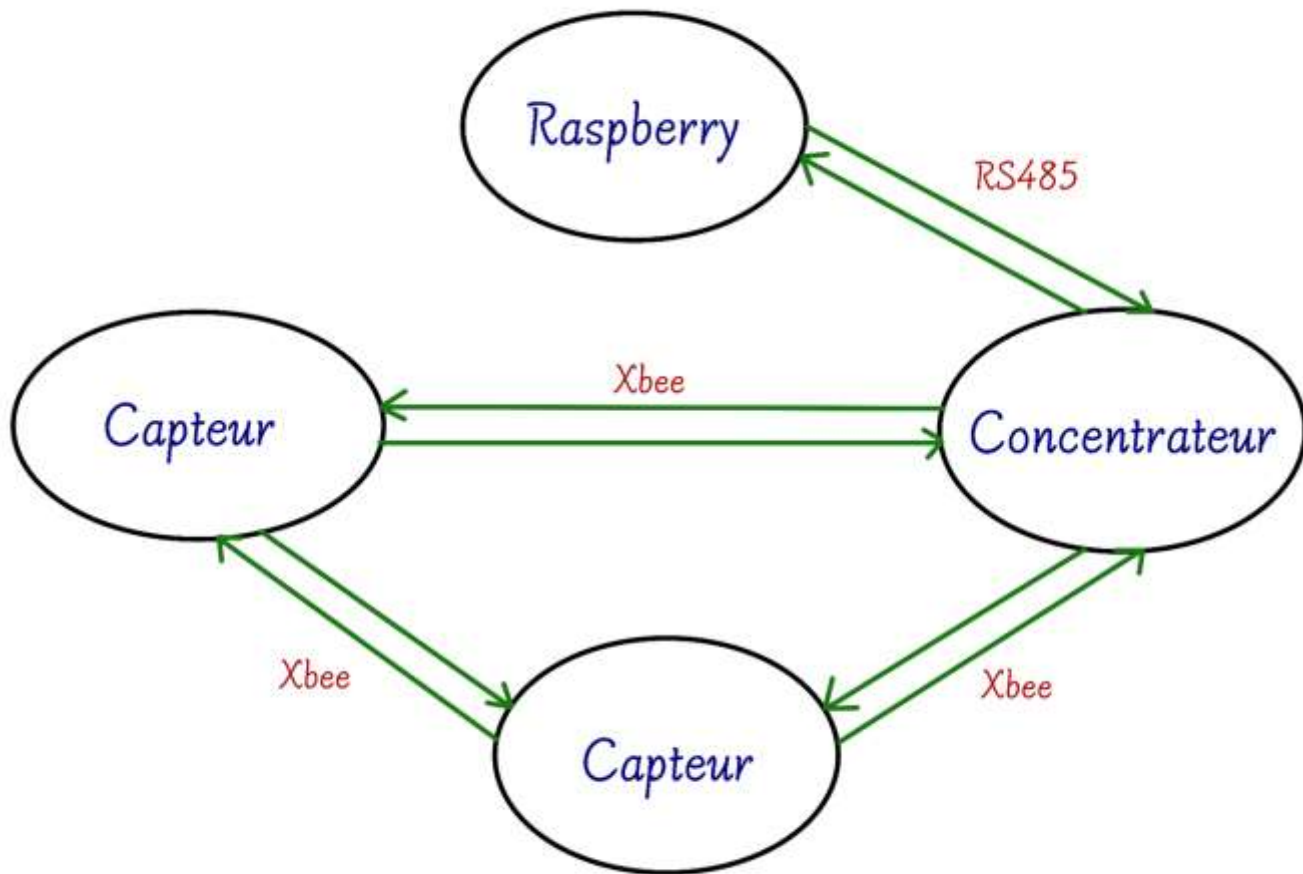


Besoins des parties prenantes



Représentation graphique

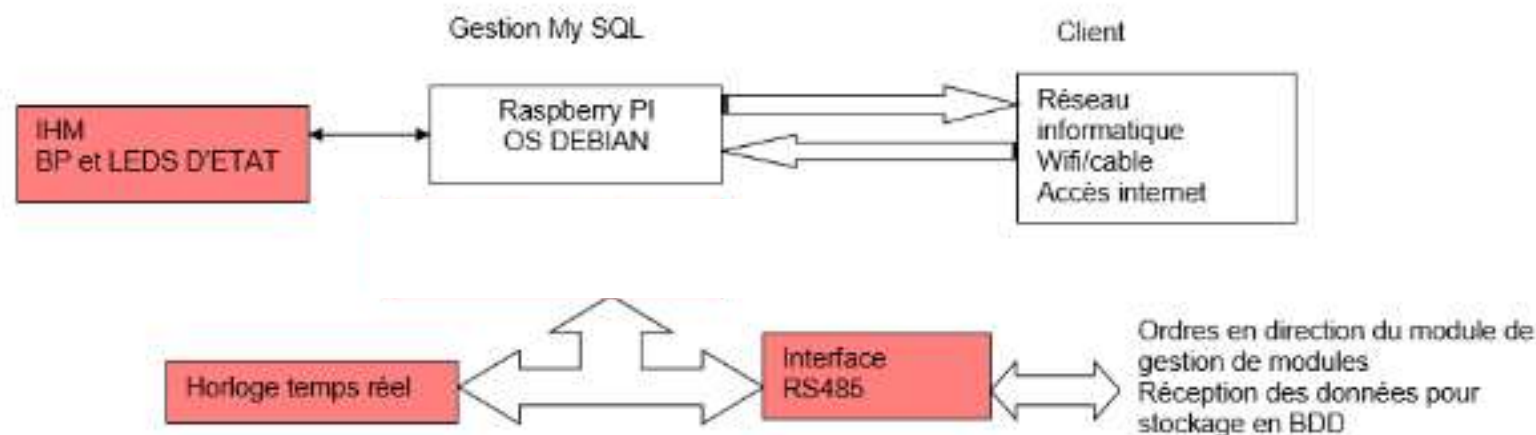




Ma partie du projet

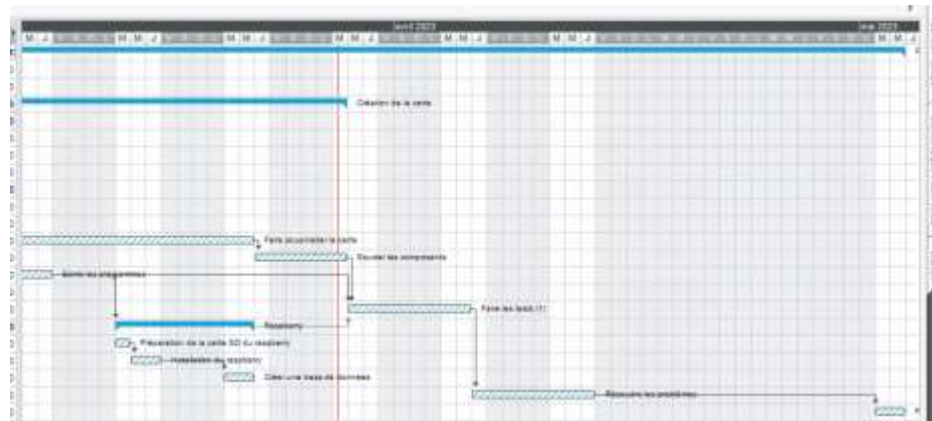
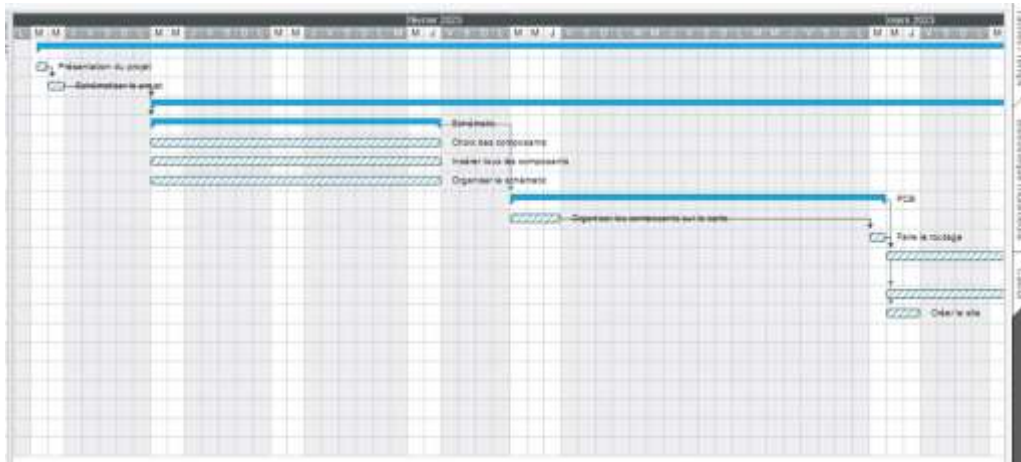
- Récupérer les informations dans une base de donnée
- Demander une mesure
- Afficher les mesures

Schéma bloc

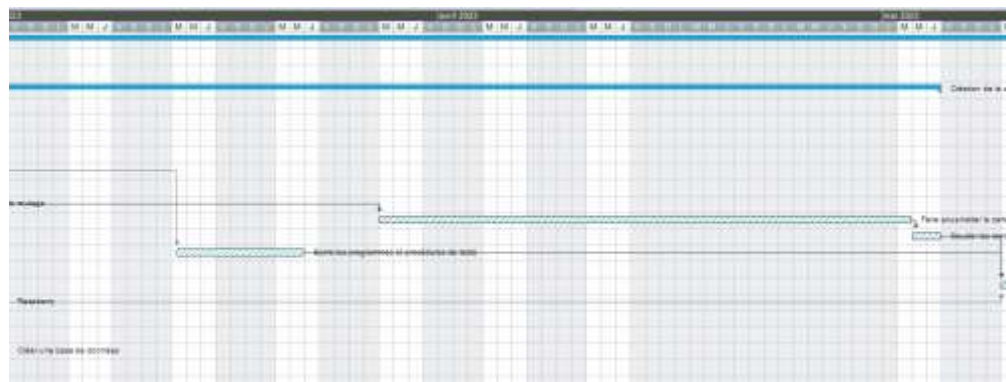
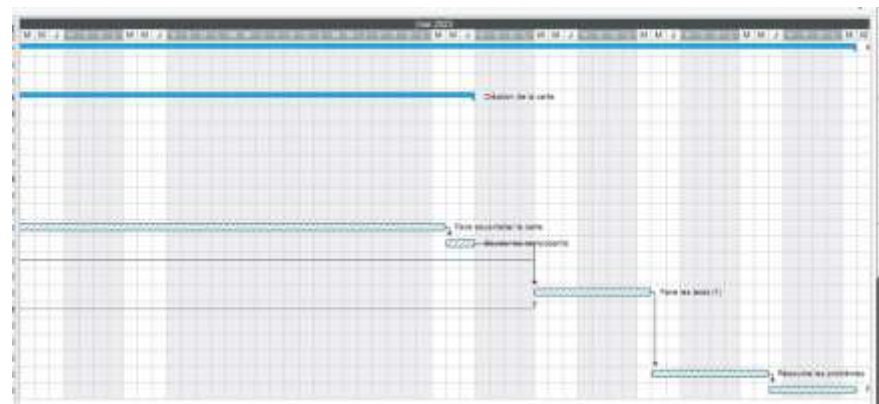
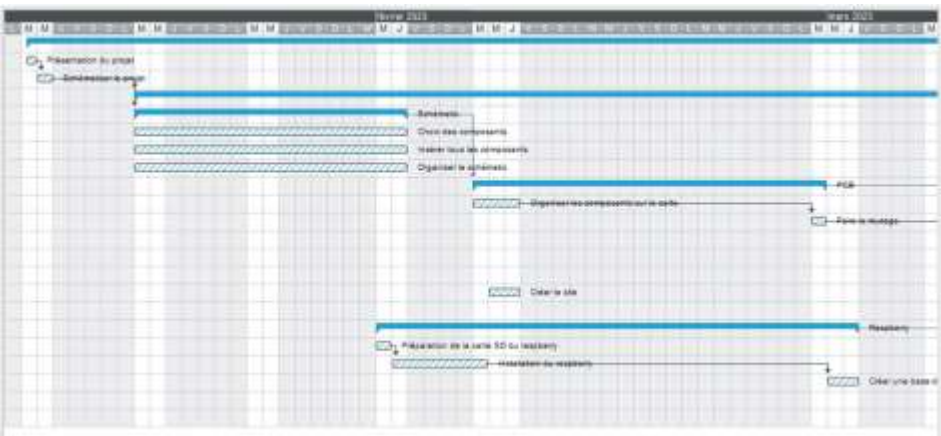


- Gérer l'IHM
- Gestion de l'horloge temps réel
- Envoyer les ordres au module de gestion
- Recevoir et stocker les données de mesure dans une Base de données
- Rendre disponible les informations de mesure à travers un réseau informatique.
- Mise à jour logiciel possible à distance à travers le réseau

Planning prévisionnel



Planning réel



**Horloge en temps réel (RTC)
STMicroelectronics série I2C,
SOIC, 5.5 V, 64B RAM, 8 broches**



Attribut	Valeur
Fonctions	Calendrier, désélection de puce, permutation
Format de données	DW.DM.M.Y
Format horaire	HH:MM:SS
RAM utilisateur	64B
Type de Bus	série I2C
Type de montage	CMS
Type de boîtier	SOIC
Nombre de broches	8
Dimensions	4.9 x 3.9 x 1.25mm
Longueur	4.9mm

Largeur	3.9mm
Hauteur	1.25mm
Tension d'alimentation fonctionnement maximum	5.5 V
Température de fonctionnement minimum	-40 °C
Température d'utilisation maximum	+85 °C
Tension d'alimentation de fonctionnement minimum	4.5 V

Horloge I2C

horloge en temps réel NXP série I2C, SOIC, 6 V, 8B RAM, 8 broches

doi:10.1017/S0022292411000445 Published online by Cambridge University Press



The growth of seed lot

Caractéristiques techniques

Attribut	Valeur
Fonctions	Alarme, Système de secours, calendrier
Format de données	DWDMR 7
Format horaire	Heure UTC
EMM utilisateur	FE
Type de bus	série RS-485
Type de montage	CMR
Type de buffet	SDS
Modèle de broches	8
Dimensions	5 x 4 x 1,43mm
Longueur	5mm
Largeur	4mm
Hauteur	1,43mm
Tension d'alimentation fonctionnellement maximale	6 V
Tension d'alimentation de fonctionnement minimale	1 V
Température de fonctionnement minimale	-40 °C
Température d'utilisation maximale	+85 °C

RS485

Numéro de pièce du fabricant	MAX3488ESA+T
Fabricant :	APPAREILS ANALOGIQUES
Code de commande :	3925790
Type d'appareil :	Émetteur-récepteur RS422 / RS485
Type d'interface CI :	RS422, RS485
Nombre de chauffeurs :	1Pilotes
Nombre de récepteurs :	1Récepteurs
Tension d'alimentation minimale :	3V
Tension d'alimentation maximale :	3.6V
Style de boîtier de pîote :	NSOIC
Nombre de broches :	8 broches
Température de fonctionnement minimale :	-40°C
Température de fonctionnement maximale :	85°C
En haut	

< Émetteurs-récepteurs RS232, RS485, RS422

APPAREILS ANALOGIQUES MAX3488ESA+T

Émetteur-récepteur , RS422, RS485, 1 pilote, 1 récepteur, 3 à 3,6 V, NSOIC-8, 85 °C

TEXAS INSTRUMENTS SN65HVD3082EDR

Transceiver RS485, 1 Driver, 200 Kbps, 4.5 V à 5.5 V, SOIC-8

Réf. Fabricant	SN65HVD3082EDR
Fabricant :	TEXAS INSTRUMENTS
Code Commande :	3007027
Type de circuit :	Émetteur-récepteur RS485
Type d'interface CI :	RS485
Nombre de drivers :	1Driver(s)
Nombre de récepteurs :	1Receivers
Tension, alimentation min.:	4.5V
Tension, alimentation max.:	5.5V
Type de boîtier CI Driver :	SOIC
Nombre de broches :	8Broche(s)
Température d'utilisation min.:	-40°C
Température de fonctionnement max.:	85°C
Gamme de produit :	-
MSL :	MSL 1 - Illimité

Régulateur de tension,
LD1117S33TR, 1.3A, SOT-223
3+Tab broches.



Régulateur de tension

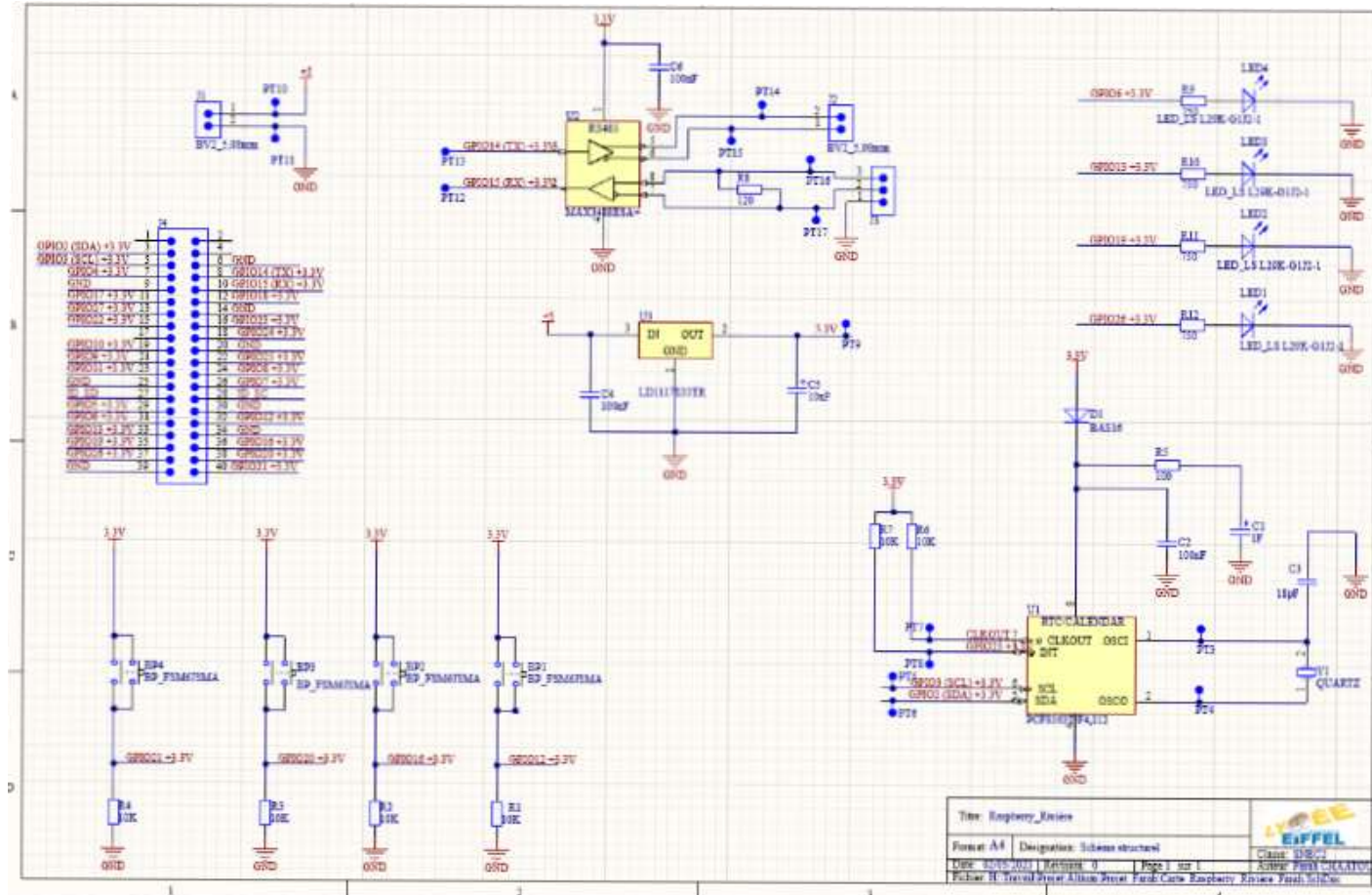
Convertisseur abaisseur de
tension CMS Texas Instruments,
600mA, 3,3 V, 5,5 V sortie
Ajustable



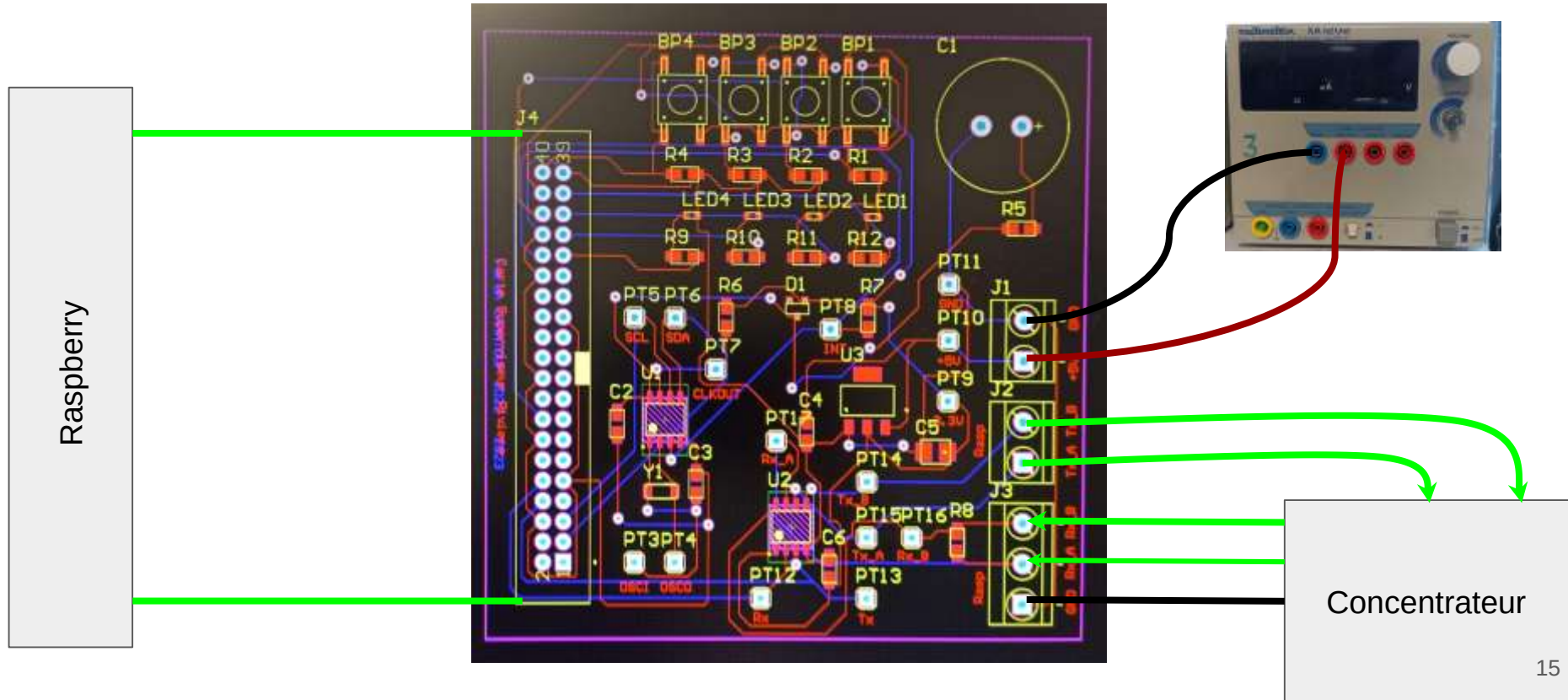
Attribut	Valeur
Courant de sortie maximum	1,3A
Type de régulateur	Tension à faible chute
	Nombre de sorties
Tension de sortie	3,3 V
Régulation en ligne	6 mV
Régulation de charge	10 mV
Précision	±1%
Polarité	Positif
Type de montage	CMS
Type de boîtier	SOT-223
Courant de repos	5mA

Attribut	Valeur
Courant de sortie maximum	600mA
Tension de sortie maximum	3,3 V
Fonction régulateur	Contrôleur Buck
Tension de sortie minimum	1 V
Tension d'entrée minimale	2,7 V
Tension d'entrée maximale	5,5 V
Type de sortie	Ajustable
Type de montage	CMS
Nombre de sorties	1
Nombre de broches	5
Fréquence de découpage maximum	2 000 kHz

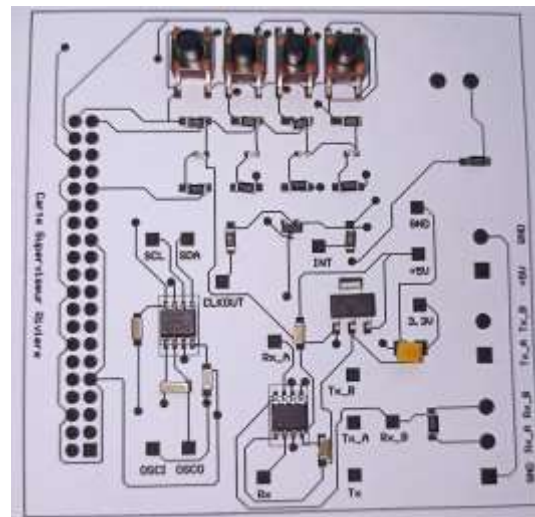
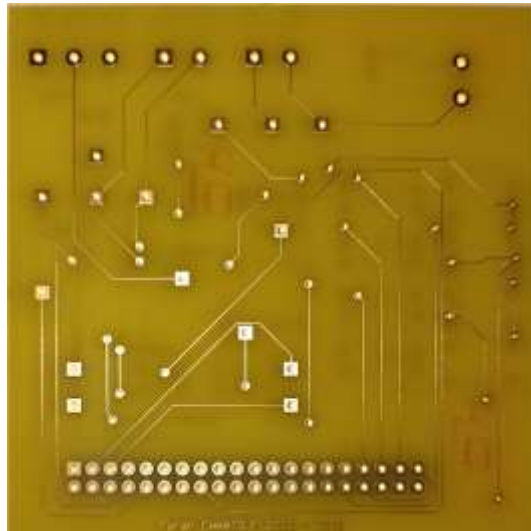
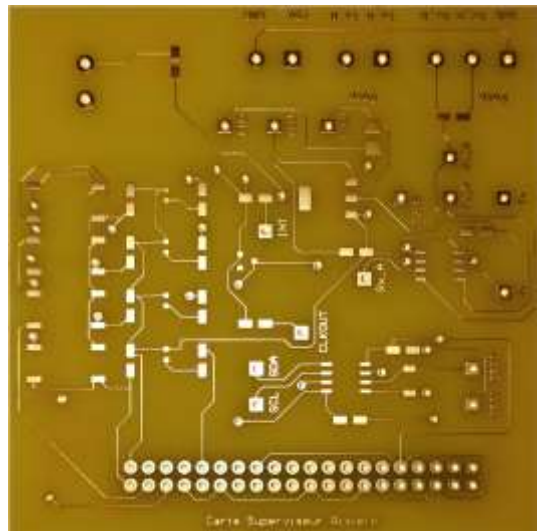
Schéma structurel de la carte

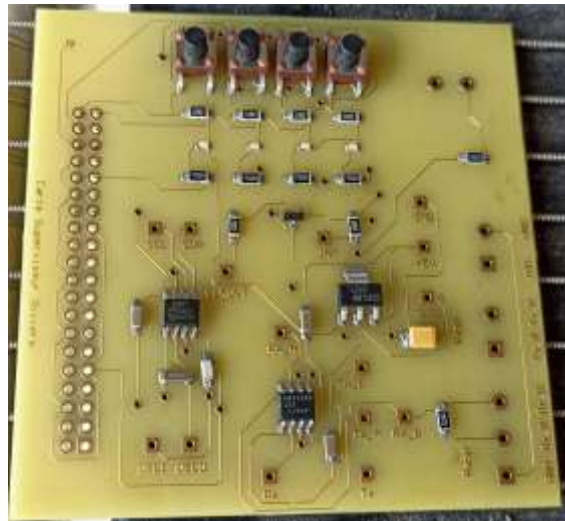
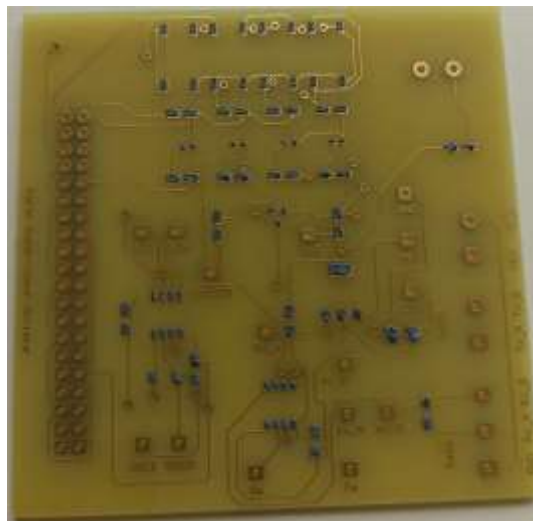
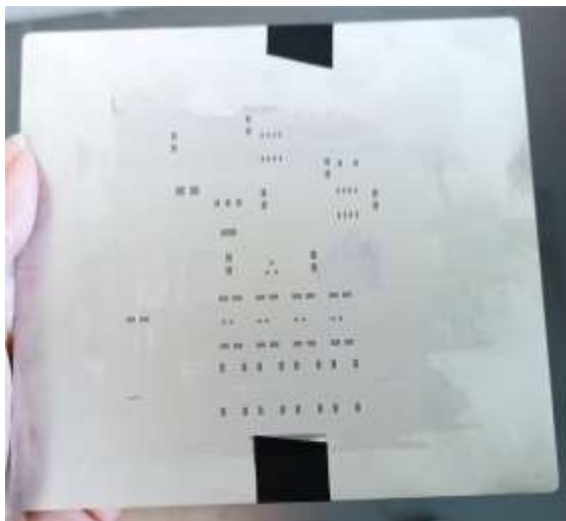


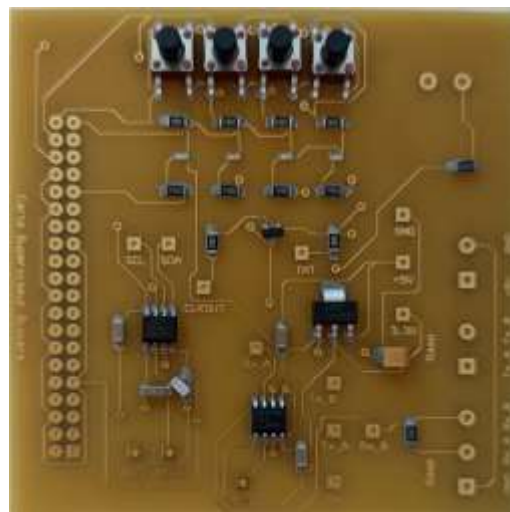
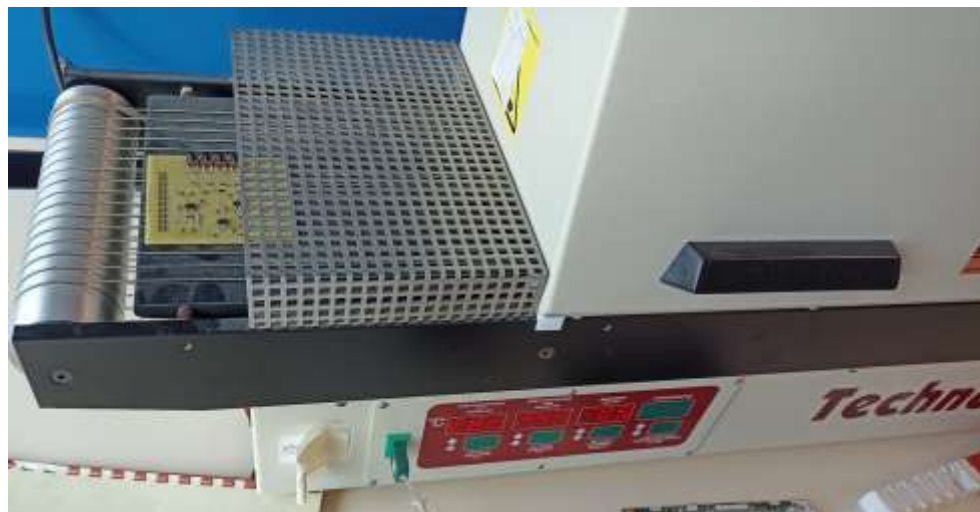
PCB et connection avec le système

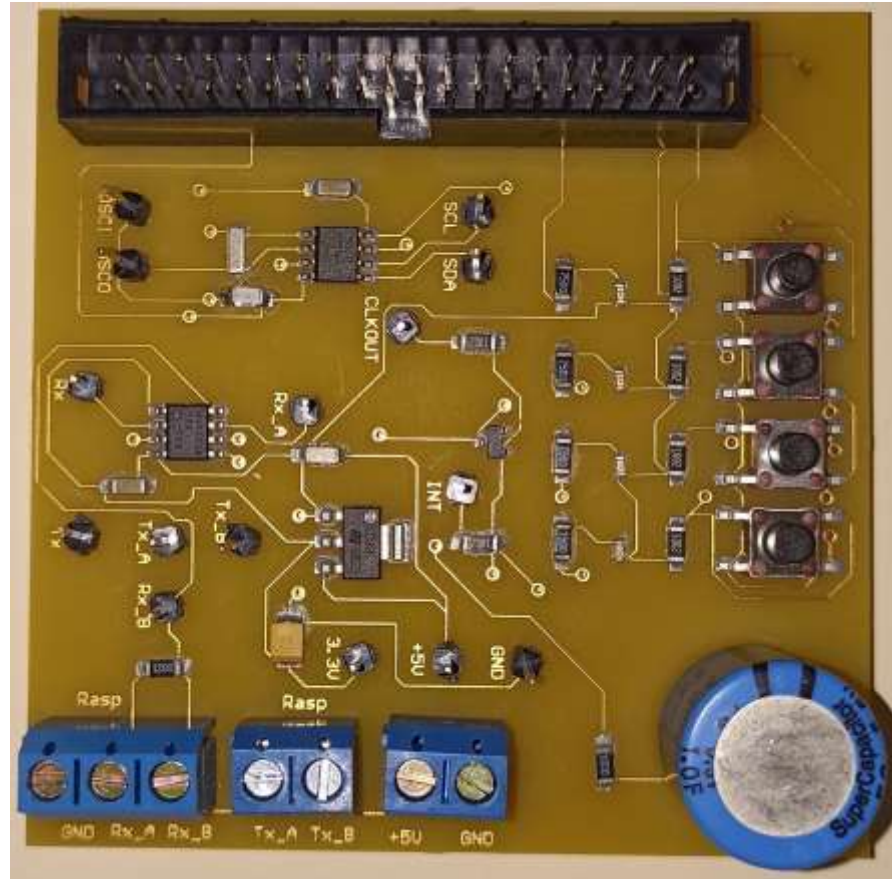


Fabrication de la carte









Tests pour les alimentations

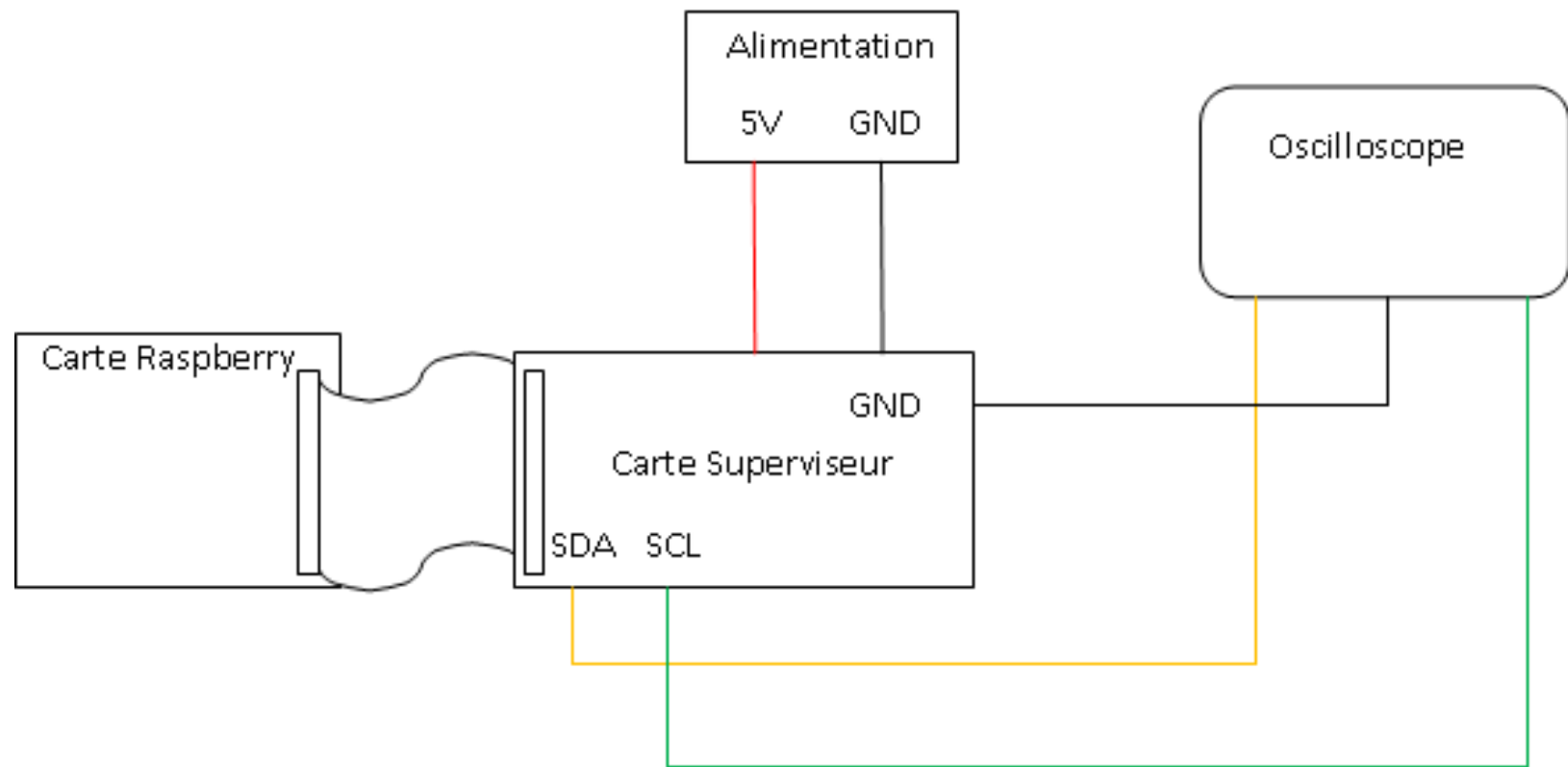
Ce dont on a besoin :

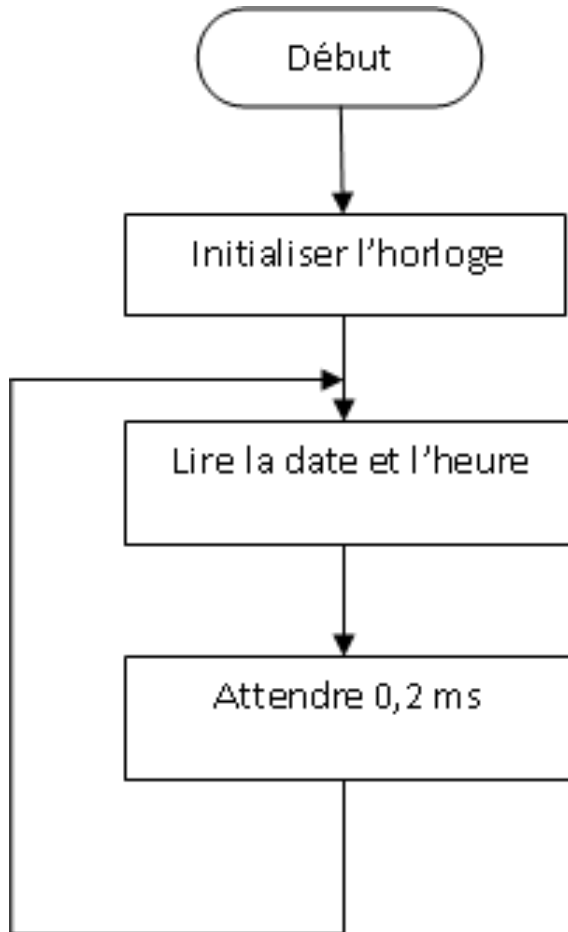
- Carte Supervision
- Alimentation 5V
- Câbles
- Multimètre

Résultat des tests pour les alimentations



Test Horloge I2C





Condition de validation :

Si l'heure s'incrémente : OK

```

import smbus
import time
Bus = smbus.SMBus(1)

i = 0x00
j = 0x00
seconde = 0x00
minute = 0x45
heure = 0x11
days = 0x10
weekdays = 0x03
mois = 0x05
annee = 0x23

# faire sudo i2cdetect -y 1 pour avoir l'adresse du bus i2c
addr = 0x51

#Bus.write_i2c_block_data ( Adresse du périphérique, Adresse du registre, [valeur1, valeur2,...] )
Bus.write_i2c_block_data (addr, 0x00, [i, j, seconde, minute, heure, days, weekdays, mois, annee] )

while True:

    #Bus.read_i2c_block_data ( adresse du périphérique, adresse du registre, bloc d'octets )
    seconde = Bus.read_byte_data ( addr, 0x02 )
    minute = Bus.read_byte_data ( addr, 0x03 )
    heure = Bus.read_byte_data ( addr, 0x04 )
    days = Bus.read_byte_data ( addr, 0x05 )
    weekdays = Bus.read_byte_data ( addr, 0x06 )
    mois = Bus.read_byte_data ( addr, 0x07 )
    annee = Bus.read_byte_data ( addr, 0x08 )

    seconde = seconde & 0b01111111
    minute = minute & 0b01111111
    heure = heure & 0b00111111
    days = days & 0b00111111
    weekdays = weekdays & 0b00000011
    mois = mois & 0b00111111

    if weekdays == 0 :
        weekdays = "Dimanche"
    elif weekdays == 1 :
        weekdays = "Lundi"
    elif weekdays == 2 :
        weekdays = "Mardi"
    elif weekdays == 3 :
        weekdays = "Mercredi"
    elif weekdays == 4 :
        weekdays = "Jeudi"
    elif weekdays == 5 :
        weekdays = "Vendredi"
    elif weekdays == 6 :
        weekdays = "Samedi"

    print(weekdays,"(0:02s) / (1:02s) / (2:02s) (3:02s) : (4:02s) : (5:02s) ".format( days, mois, annee, heure, minute, seconde))
    time.sleep(1)

```

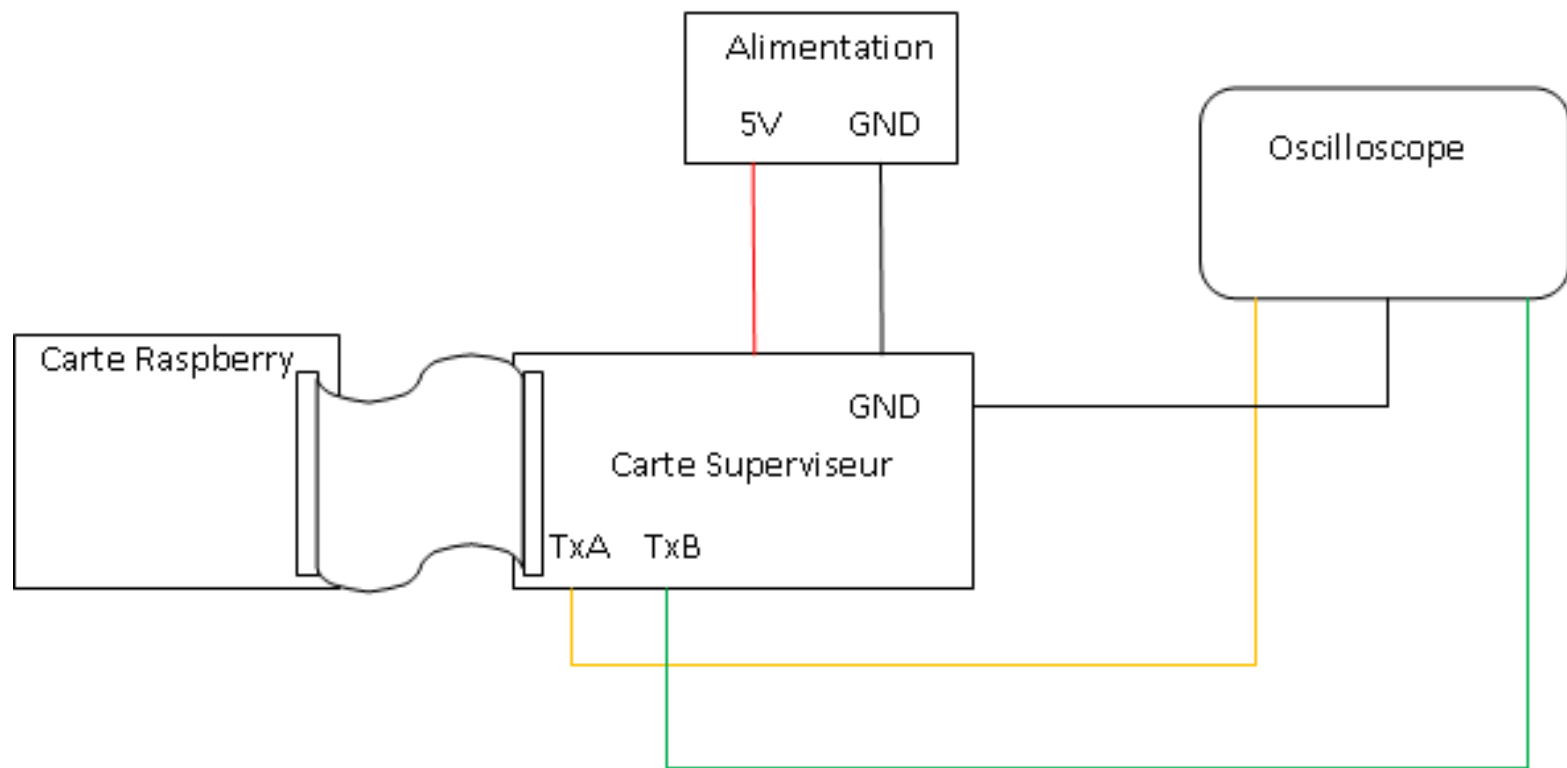
Résultat du test de l'horloge I2C

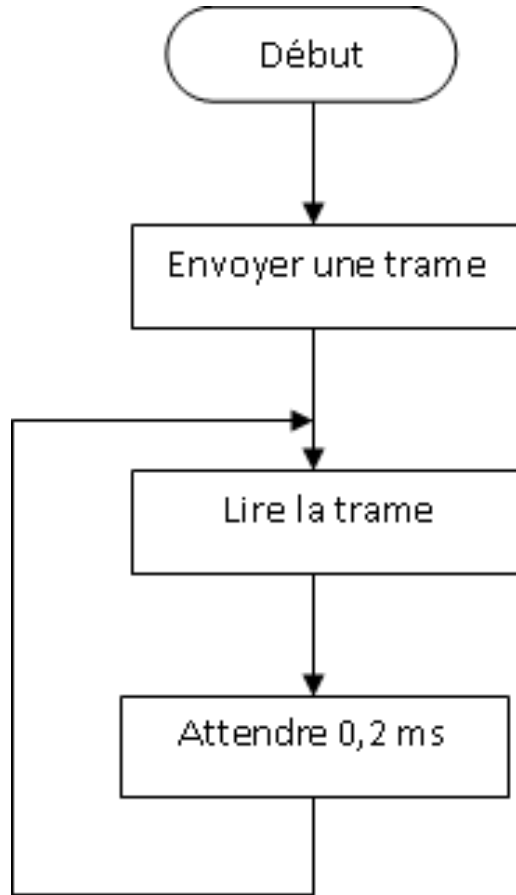
```

pi@raspberrypi:~ $ python Horloge.py
Mercredi 10 / 05 / 23 11 : 00 : 00
Mercredi 10 / 05 / 23 11 : 00 : 01
Mercredi 10 / 05 / 23 11 : 00 : 02
Mercredi 10 / 05 / 23 11 : 00 : 03
Mercredi 10 / 05 / 23 11 : 00 : 04
Mercredi 10 / 05 / 23 11 : 00 : 05
Mercredi 10 / 05 / 23 11 : 00 : 06
Mercredi 10 / 05 / 23 11 : 00 : 07
Mercredi 10 / 05 / 23 11 : 00 : 08
Mercredi 10 / 05 / 23 11 : 00 : 09
Mercredi 10 / 05 / 23 11 : 00 : 10
Mercredi 10 / 05 / 23 11 : 00 : 11
Mercredi 10 / 05 / 23 11 : 00 : 12
Mercredi 10 / 05 / 23 11 : 00 : 13
Mercredi 10 / 05 / 23 11 : 00 : 14
Mercredi 10 / 05 / 23 11 : 00 : 15
Mercredi 10 / 05 / 23 11 : 00 : 16

```

Test RS485





Condition de validation :

Si la trame est correcte sur les deux voies de l'oscilloscope : OK

Résultat du test RS485

```
import serial
import RPi.GPIO as GPIO
import time
```

```
port = serial.Serial("/dev/ttyAMA0", baudrate=115200, timeout=3.0) ;
```

```
#Transmettre la trame
```

```
while True :
```

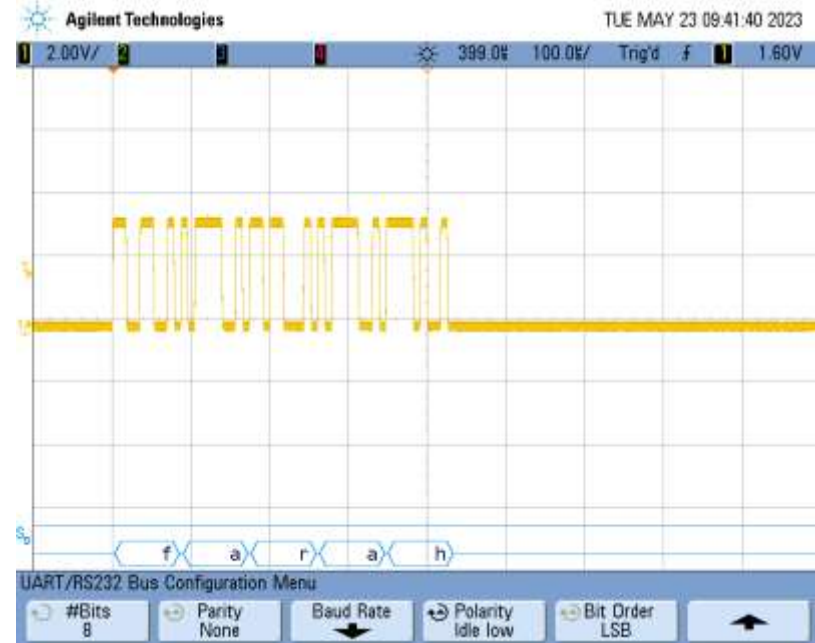
```
    port.write(b"farah")
```

```
    #rcv = port.read(10)
```

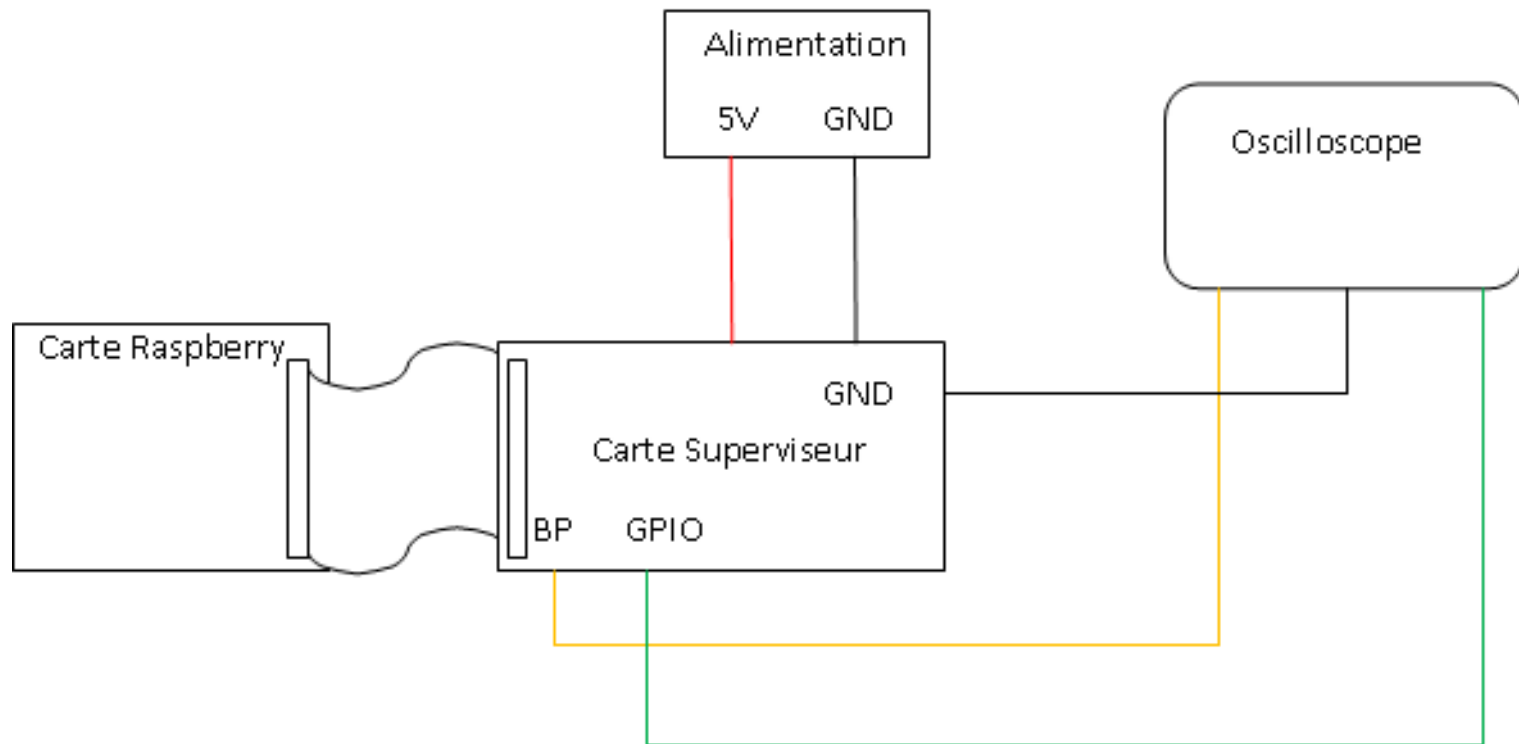
```
    # lit 10 caracteres
```

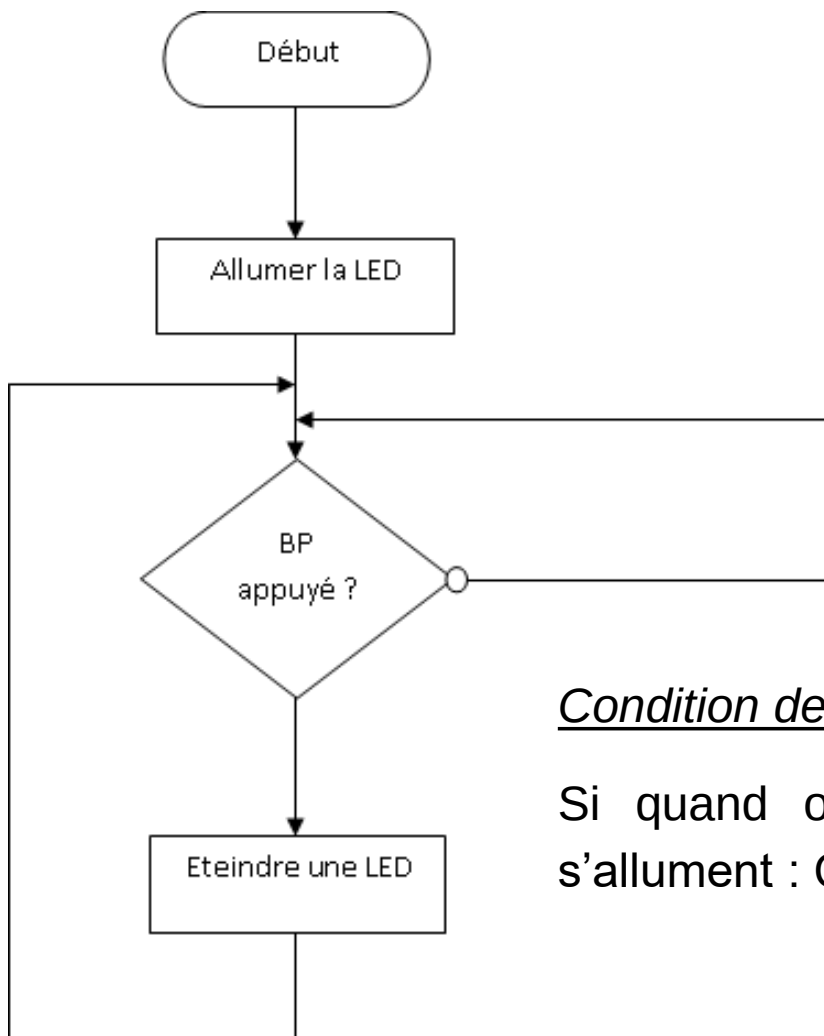
```
    #print(rcv)
```

```
    time.sleep(0.5)
```



Test BP et LED





Condition de validation :

Si quand on appuie sur les 4 BP les 4 LED s'allument : OK

Résultat du test pour les PB et LED

```
# Import des modules
```

```
import RPi.GPIO as GPIO
import time
```

```
GPIO.setwarnings(False)
```

```
# Initialisation de la numérotation et des E/S
GPIO.setmode(GPIO.BCM)
```

```
GPIO.setup(26, GPIO.OUT, initial = GPIO.LOW) # LED (26;19;13;6)
GPIO.setup(12, GPIO.IN) # BP (12;16;20;21)
```

```
GPIO.setup(19, GPIO.OUT, initial = GPIO.LOW) # LED (26;19;13;6)
GPIO.setup(16, GPIO.IN) # BP (12;16;20;21)
```

```
GPIO.setup(13, GPIO.OUT, initial = GPIO.LOW) # LED (26;19;13;6)
GPIO.setup(20, GPIO.IN) # BP (12;16;20;21)
```

```
GPIO.setup(6, GPIO.OUT, initial = GPIO.LOW) # LED (26;19;13;6)
GPIO.setup(21, GPIO.IN) # BP (12;16;20;21)
```

```
# Une LED s'allume
```

```
# On appuie sur le bouton pour éteindre la LED puis c'est à la LED
```

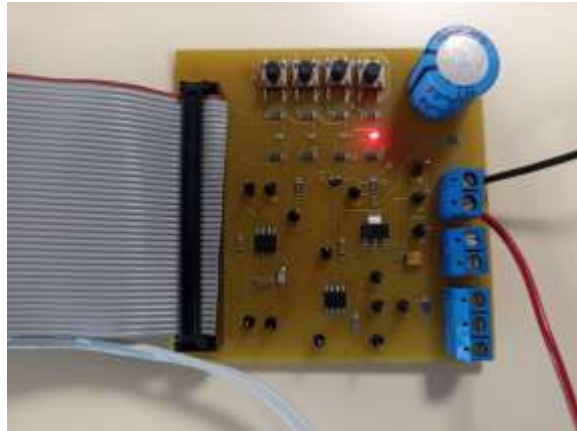
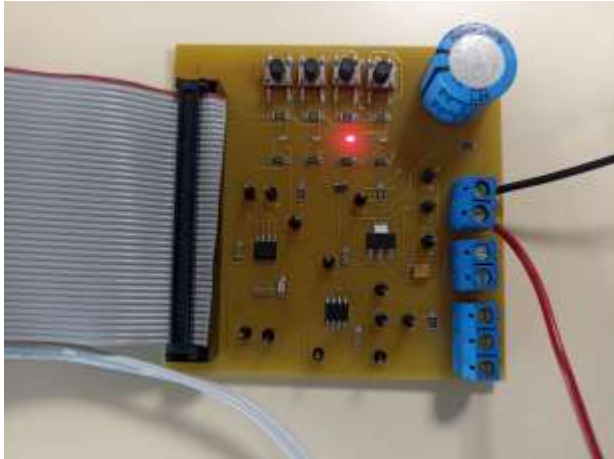
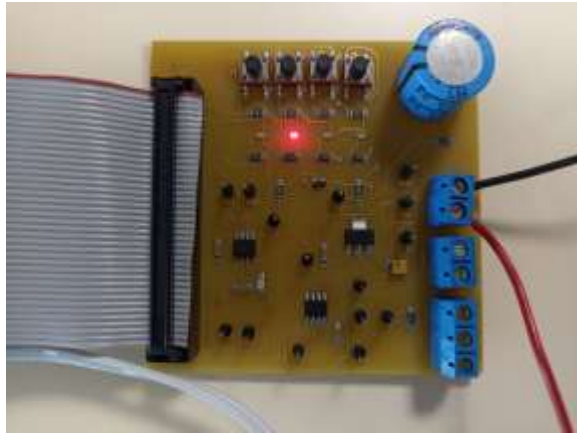
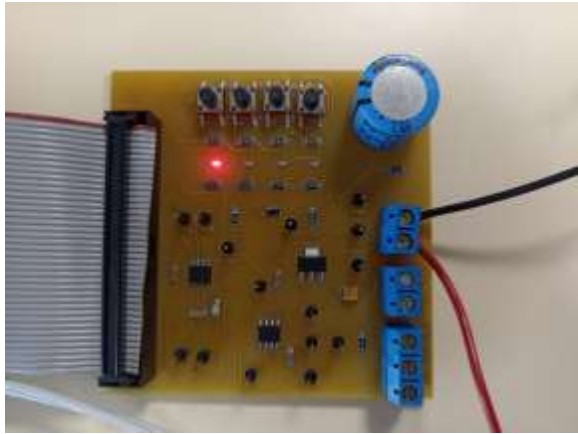
```
while True:
```

```
    state = GPIO.input(21)
    if not state:
        GPIO.output(6, GPIO.HIGH)
        while not state:
            state = GPIO.input(21)
            time.sleep(0.02)
        GPIO.output(6, GPIO.LOW)
        time.sleep(0.02)
```

```
    state = GPIO.input(20)
    if not state:
        GPIO.output(13, GPIO.HIGH)
        while not state:
            state = GPIO.input(20)
            time.sleep(0.02)
        GPIO.output(13, GPIO.LOW)
        time.sleep(0.02)
```

```
    state = GPIO.input(16)
    if not state:
        GPIO.output(19, GPIO.HIGH)
        while not state:
            state = GPIO.input(16)
            time.sleep(0.02)
        GPIO.output(19, GPIO.LOW)
        time.sleep(0.02)
```

```
    state = GPIO.input(12)
    if not state:
        GPIO.output(26, GPIO.HIGH)
        while not state:
            state = GPIO.input(12)
            time.sleep(0.02)
        GPIO.output(26, GPIO.LOW)
        time.sleep(0.02)
```



Code du site web

```

1  <DOCTYPE html>
2  <html>
3
4  <head>
5
6      <meta charset="utf-8" />
7      <link rel="stylesheet" href="css/text.css" href="css/css3.css" />
8      <title>Balade dans le ruisseau</title>
9
10 </head>
11
12 <body>
13     <div id="titre">
14         <h1>Balade dans le ruisseau</h1>
15     </div>
16
17     <div id="contenu">
18         <p>Sur cette page vous pourrez retrouver les mesures effectuées dans les ruisseaux des alentours.</p>
19     </div>
20
21 </body>
22 </html>

```

```

101 catch(Exception $e)
102 {
103     //Envoye le message d'erreur et il se gèreMessage()
104     die("Erreur : ". $e->getMessage());
105 }
106
107 //Assigne les éléments de la MTD à la variable réponse
108 $response = $bdd->query("SELECT * FROM Captures");
109 if($response == false)
110 {
111     die("Erreur");
112 }
113
114 }
115
116 }
117
118 //Assigne à $ligne les valeurs de réponse
119 while($ligne = $response->fetch(PDO::FETCH_ASSOC))
120 {
121
122     <!--Affiche les résultats dans un tableau-->
123     <table>
124
125         <tr>
126             <td><?php echo htmlspecialchars($ligne["Date"]); </td>
127             <td><?php echo htmlspecialchars($ligne["Date"]); </td>
128             <td><?php echo htmlspecialchars($ligne["Heure"]); </td>
129             <td><?php echo htmlspecialchars($ligne["Type"]); </td>
130             <td><?php echo htmlspecialchars($ligne["Valeur"]); </td>
131         </tr>
132
133         <tr>
134             <td colspan="5"><?php echo $table->__toString();>
135         </td>
136     </tr>
137
138     <tr>
139         <td colspan="5"><strong>Terah CHATREFF</strong></td>
140     </tr>
141
142     <tr>
143         <td colspan="5"><strong>Anthony VANNANNE</strong></td>
144     </tr>
145
146     <tr>
147         <td colspan="5"><strong>Yusef ELMIL</strong></td>
148     </tr>
149
150     <tr>
151         <td colspan="5"><strong>Moah DEBROCK</strong></td>
152     </tr>
153
154     </tbody>
155 </table>

```

```
-<?php
    $bdd = new PDO('mysql:host=localhost;dbname=RIVIERE', 'root', 'rasp-user');//c
    $reponse = $bdd->query( "UPDATE Ordre SET Num_Ordre = '1' WHERE ID_Ordre = 0")
?>

<script type="text/javascript">
    alert('Vous allez être redirigé vers la page d accueil');
    document.location.href = 'Site_internet.php';
</script>
```

Site internet

← → ↻ 🔒 Non sécurisé | 172.16.19.199/Site_internet.php

Applications

Relevés des capteur des rivières

Sur cette page vous pouvez retrouver les mesures effectués dans les rivières des alentours

[Prendre les 3 mesures](#)[Récupérer la turbidité](#)[Récupérer le pH](#)[Récupérer la température](#)

=====

Jour	Date	Heure	Type	Valeur
Mercredi	2023-05-31	10:39:47	Turbidité	1
Jeudi	2023-06-01	08:24:31	Turbidité	6
Jeudi	2023-06-01	08:25:32	Turbidité	6
Jeudi	2023-06-01	08:31:35	Turbidité	8
Jeudi	2023-06-01	09:31:46	Turbidité	8
Jeudi	2023-06-01	09:44:04	Turbidité	8
Jeudi	2023-06-01	09:46:38	Turbidité	8
Jeudi	2023-06-01	09:46:38	pH	8
Jeudi	2023-06-01	09:46:38	Temperature	8
Jeudi	2023-06-01	09:49:09	Turbidité	8
Jeudi	2023-06-01	09:49:09	pH	2
Jeudi	2023-06-01	09:49:09	Temperature	4
Jeudi	2023-06-01	10:29:14	Turbidité	8
Jeudi	2023-06-01	16:10:31	Temperature	4

ÉRIC CHATAIGNER
Anthony VANMARCKE
Nicolas DUBUIT

BDD

```
pi@raspberrypi1:~ $ python test_ordre.py
```

```
0
0
0
0
0
0
0
1
1
0
23
2023
0
0
0
0
0
0
0
3
3
0
```

		ID_Ordre	Num_Ordre
☐	Éditer	0	0

Programme qui gère les ordres

```
1 import mysql.connector as mysql
2 import RPi.GPIO as GPIO
3 import time
4 import serial
5 import smbus
6 import time
7
8 Bus = smbus.SMBus(1)
9
10 # faire sudo i2cdetect -y 1 pour avoir l'adresse du bus I2C
11 addr = 0x51
12
13 GPIO.setwarnings(False)
14 # Initialisation des LED
15 GPIO.setmode(GPIO.BCM)
16
17 GPIO.setup(24, GPIO.OUT, initial = GPIO.LOW) # LED (28:19:13:6)
18 GPIO.setup(15, GPIO.OUT, initial = GPIO.LOW) # LED (26:19:13:6)
19 GPIO.setup(13, GPIO.OUT, initial = GPIO.LOW) # LED (26:19:13:6)
20 GPIO.setup(6, GPIO.OUT, initial = GPIO.LOW) # LED (26:19:13:6)
21
22 # le programme redémarre toutes les 3s
23 port = serial.Serial("/dev/ttyS0A0", baudrate=115200, timeout=3.0)
24
25 rcr = 0
26
27 while True :
28     #connection à la base de donnée
29     db = mysql.connector (host = "localhost", user = "root", password = "rasp-user", database = "BIVIERX")
30
31     #créer un curseur "cur" de base de données pour effectuer des opérations SQL
32     cur = db.cursor()
33
34     cur.execute("SELECT Num_Ordre FROM Ordre;")
35     ligne = cur.fetchall()
36     for var in ligne:
37         var_ordre=var[0]
38         #valider l'instruction SQL dans la BDD
39         db.commit()
40
41     # on ferme la connexion à la base de données comme on a fini, et pour pouvoir se reconnecter sans problème à la boucle suivante
42     db.close()
43     print(var_ordre)
44     time.sleep(1)
45
46     if var_ordre == 0 :
47         print(var_ordre)
48         var_ordre = 0
49
```

```

49
50 elif var_ordre == 1 :
51     print(var_ordre)
52     GPIO.output(26, GPIO.HIGH)
53     time.sleep(1)
54     GPIO.output(26, GPIO.LOW)
55
56     port.write(b"1")
57
58     db = mysql.connect (host = "localhost", user = "root", password = "rasp-user", database = "RIVIERE")
59     cur = db.cursor()
60     cur.execute("UPDATE Ordre SET Num_Ordre = 0 WHERE ID_Ordre = 0;")
61     var_ordre = 0
62     db.commit()
63     db.close()
64     print(var_ordre)
65
66     #Bus.read_i2c_block_data ( adresse du périphérique, adresse du registre, bloc d'octets )
67     seconde = Bus.read_byte_data ( addr, 0x02 )
68     minute = Bus.read_byte_data ( addr, 0x03 )
69     heure = Bus.read_byte_data ( addr, 0x04 )
70     days = Bus.read_byte_data ( addr, 0x05 )
71     weekdays = Bus.read_byte_data ( addr, 0x06 )
72     mois = Bus.read_byte_data ( addr, 0x07 )
73     annee = Bus.read_byte_data ( addr, 0x08)
74
75     seconde = seconde & 0b01111111
76     minute = minute & 0b01111111
77     heure = heure & 0b00111111
78     days = days & 0b00111111
79     weekdays = weekdays & 0b00000111
80     mois = mois & 0b00011111
81     annee = annee & 0b11111111
82
83     if weekdays == 0 :
84         weekdays = "Dimanche"
85     elif weekdays == 1 :
86         weekdays = "Lundi"
87     elif weekdays == 2 :
88         weekdays = "Mardi"
89     elif weekdays == 3 :
90         weekdays = "Mercredi"
91     elif weekdays == 4 :
92         weekdays = "Jeudi"
93     elif weekdays == 5 :
94         weekdays = "Vendredi"
95     elif weekdays == 6 :
96         weekdays = "Samedi"
97
98     print(annee)
99     annee = annee + 2000
00     print(annee)

```

```

104
105 elif var_ordre == 2 :
106     print(var_ordre)
107     GPIO.output(19, GPIO.HIGH)
108     time.sleep(1)
109     GPIO.output(19, GPIO.LOW)
110
111     port.write(b"2")
112
113     db = mysql.connect (host = "localhost", user = "root", password = "rasp-user", database = "RIVIERE")
114     cur = db.cursor()
115     cur.execute("UPDATE Ordre SET Num_Ordre = 0 WHERE ID_Ordre = 0;")
116     var_ordre = 0
117     db.commit()
118     db.close()
119
120
121     #rcv = port.read(8)
122     rcv = 8
123
124     valeurs = (weekdays, "{0:04d}-{1:02x}-{2:02x}".format(annee,mois,days), "{0:02x}:{1:02x}:{2:02x}".format(heure, minute, seconde),rcv)
125
126     print(valeurs)
127     db = mysql.connect (host = "localhost", user = "root", password = "rasp-user", database = "RIVIERE")
128     cur = db.cursor()
129     cur.execute("INSERT INTO Capteur VALUES (NULL, %s, %s, %s, 'Turbidite' ,'%s');", valeurs)
130     db.commit()
131     db.close()

```

+ Options

↔ T ↔ ▼						ID	Jour	Date	Heure	Type	Valeur	
☐		Éditer		Copier		Supprimer	38	Mardi	2023-06-06	11:51:17	Temperature	4
☐		Éditer		Copier		Supprimer	39	Mardi	2023-06-06	11:52:57	Turbidite	8
☐		Éditer		Copier		Supprimer	40	Mardi	2023-06-06	13:57:45	Turbidite	8
☐		Éditer		Copier		Supprimer	41	Mardi	2023-06-06	13:59:05	pH	2
☐		Éditer		Copier		Supprimer	42	Mardi	2023-06-06	14:28:00	Temperature	4

Conclusion

Fait :

- Conception de la carte
- Test du bon fonctionnement de la carte
- Test avec la carte concentrateur
- Gérer les ordres
- Ajouter les relevés à une BDD
- Affichage des données sur la page web

A faire :

- Réception des valeurs réels des capteurs
- Test complet du système